EdgeFed-Bench v2: Empirical Robustness and Precision Trade-offs in Byzantine-Tolerant Federated Learning

A 280-Configuration Factorial Study Across Seeds, Attacks, Quantization and Robust Defences

Author: Raj Chhapariya | **Repository:** github.com/Rajchhapariya/edgefed-bench | **Date:** October 2026

GRID SCOPE	UPLINK SAVINGS	MULTI-KRUM AT 30%	ASSUMED F = 2
320 Runs	**-75.0%**	**84.75%**	**51.65%**
280 grid + 40 wrong-f across 5 seeds	397.5 KB to 99.4 KB / client / round	final accuracy, sign-flip, INT8	Multi-Krum accuracy, actual f = 6

1. Problem Formulation & Research Question

Federated learning coordinates distributed machine learning training across decentralized edge devices while preserving raw data locality. In production edge deployments, two structural constraints collide:

1. **Uplink Bandwidth Asymmetry:** Edge devices communicate across bandwidth-constrained, metered cellular and IoT channels where upstream transmission is substantially slower and costlier than downstream reception.
2. **Byzantine Vulnerability:** Decentralized edge topologies allow rogue, compromised, or malfunctioning clients to transmit poisoned gradient updates. Standard Federated Averaging (FedAvg) aggregates updates through linear averaging, rendering it provably vulnerable to even a single arbitrary outlier.

Gradient quantization reduces uplink communication costs by mapping continuous float32 parameter deltas into low-bitwidth integers. However, lossy quantization injects stochastic truncation noise. This investigation addresses the central empirical research question:

Does uniform symmetric INT8 gradient quantization degrade the mathematical outlier separation distance and convergence guarantees of Byzantine-tolerant aggregators (Multi-Krum, Coordinate-wise Median, Coordinate-wise Trimmed Mean) under severe non-IID Dirichlet label skew ($\alpha = 0.5$)?

2. Experimental Setup & System Architecture

2.1. Dataset and Non-IID Dirichlet Partitioning

Experiments execute on Fashion-MNIST (60,000 training samples, 10,000 test samples, 10 classes). To simulate real-world heterogeneous edge deployments, training samples are partitioned across $N = 20$ edge clients using a Dirichlet distribution with concentration parameter $\alpha = 0.5$. Each client receives a skewed subset of labels, creating substantial class imbalance across clients. To guarantee reproducibility, partitions are serialized into independent Apache Parquet shards per seed (`data/clients_seed_{seed}/client_{id}.parquet`) using Snappy compression and queried via an in-memory DuckDB columnar engine (`EdgeStore`).

2.2. Edge Classifier and Local Optimization

Each edge client trains an `EdgeClassifier` Multi-Layer Perceptron (784-dimensional input, 128 hidden units with ReLU activation, 10 output logits; 101,770 total trainable parameters). Local optimization uses stateless Stochastic Gradient Descent (SGD) with learning rate $\eta = 0.05$, momentum 0.9, batch size 128, and $E = 1$ local epoch per communication round. In each round t, client i computes its local parameter update:

$$\Delta \boldsymbol{w}_i^t = \boldsymbol{w}_{i,\,\text{local}}^t - \boldsymbol{w}_{\text{global}}^t$$

2.3. Uniform Symmetric INT8 Quantization

Under INT8 wire precision, each client compresses its continuous update vector $\boldsymbol{x} \in \mathbb{R}^D$ into signed 8-bit integers using symmetric scaling:

$$\alpha = \max_j |x_j|, \quad S = \frac{\alpha}{127}, \quad q_j = \text{clip}\left(\text{round}\left(\frac{x_j}{S}\right), -127, 127\right) \in \mathbb{Z}_{\text{int8}}$$

Reconstruction on the server yields $\hat{x}_j = q_j \cdot S$. The element-wise quantization error is bounded by $|x_j - \hat{x}_j| \leq \frac{S}{2} = \frac{\alpha}{254}$. Payload per client per round drops from 407,080 bytes ($101,770 \times 4$ bytes float32) to 101,774 bytes ($101,770 \times 1$ byte int8 $+4$ bytes float32 scale), a 75.0% uplink reduction.

2.4. Byzantine Adversarial Attack Vectors

The benchmark evaluates two primary attack vectors deployed by a designated fraction $\frac{f}{N} \in \{10\%, 20\%, 30\%\}$ of clients:

- **Sign-Flip Attack:** Adversaries invert their true gradient direction and scale its magnitude: $\boldsymbol{v}_{\text{byz}} = -2.5 \cdot \boldsymbol{v}_{\text{honest}}$. This directly steers the server model away from optimal empirical minima toward higher loss regimes.
- **Gaussian Noise Injection:** Adversaries transmit uncorrelated Gaussian noise vectors: $\boldsymbol{v}_{\text{byz}} \sim \mathcal{N}(0, \sigma^2 \boldsymbol{I})$ with $\sigma = 5.0$, inducing variance explosion in central updates.

2.5. Robust Aggregation Algorithms

- **FedAvg (Baseline):** Computes standard coordinate-wise arithmetic mean: $\boldsymbol{v}_{\text{agg}} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{v}_i$.
- **Multi-Krum (Blanchard et al., NeurIPS 2017):** Computes pairwise squared Euclidean distances $d_{ij}^2 = \|\boldsymbol{v}_i - \boldsymbol{v}_j\|_2^2$ in float64 precision. For each update, it sums distances to its $N - f - 2$ closest neighbors to establish a Krum score $s(i)$. It selects the $m = N - f$ updates with the lowest scores and averages them. Multi-Krum requires $2f + 2 < N$. For $N = 20$ and $f = 6$ (30% attackers), $2(6) + 2 = 14 < 20$, strictly satisfying the theoretical bound.
- **Coordinate-wise Median:** Computes the median independently along each parameter dimension: $\left[\boldsymbol{v}_{\text{agg}}\right]_j = \text{median}\left(\left\{[\boldsymbol{v}_1]_j, \ldots, [\boldsymbol{v}_N]_j\right\}\right)$. Resilient up to a 50% breakdown point.
- **Coordinate-wise Trimmed Mean:** For each coordinate j, sorts candidate values, excludes the $k = \lceil \beta N \rceil$ largest and k smallest values (where $\beta = \frac{f}{N}$), and averages the remaining $N - 2k$ coordinates. Requires $2k < N$.

2.6. Full Factorial Experiment Grid

The experimental grid sweeps 280 configurations: 4 Aggregators × 2 Attacks (sign-flip, Gaussian) × 3 Attacker Shares (10%, 20%, 30%) × 2 Wire Precisions (Float32, INT8) × 5 Seeds (0, 1, 2, 3, 4) across 50 rounds, plus 40 clean baseline runs (0% attackers). A single seed controls Dirichlet partitioning, model weight initialization, Byzantine node selection, Gaussian attack noise, and client-level batch shuffling, guaranteeing exact bit-level determinism on CPU.

3. Empirical Results & Comparative Analysis

Attack Vector	Share	FedAvg	Multi-Krum	Coord Median	Trimmed Mean
None (Clean)	0%	85.30 +- 0.31%	85.35 +- 0.25%	84.65 +- 0.44%	85.36 +- 0.27%
Gaussian	10%	48.79 +- 3.51%	85.30 +- 0.39%	84.54 +- 0.35%	85.11 +- 0.40%
Gaussian	20%	41.79 +- 6.24%	85.17 +- 0.32%	84.64 +- 0.34%	84.89 +- 0.22%
Gaussian	30%	42.81 +- 4.09%	85.13 +- 0.29%	84.73 +- 0.50%	84.87 +- 0.39%
sign-flip	10%	78.79 +- 3.58%	85.28 +- 0.38%	83.96 +- 0.63%	84.11 +- 0.65%
sign-flip	20%	41.60 +- 29.28%	84.90 +- 0.26%	82.97 +- 0.62%	83.00 +- 0.45%
sign-flip	30%	10.00 +- 0.00%	84.75 +- 0.48%	80.04 +- 0.30%	80.37 +- 0.30%

Core Result: Quantization Noise Invariance under Severe Non-IID Skew

Across the 240 attacked configurations, switching from Float32 to INT8 changed Multi-Krum final accuracy at 20% sign-flip from 84.89% to 84.90%, a difference smaller than one standard deviation (0.26). Trimmed Mean reaches 83.00% in the same setting. Per-element INT8 error is bounded by S/2; a likely reason the defences are unaffected is that this error is small next to the distance between honest and attacker updates, but that distance was not measured here, so this is a hypothesis.

3.1. Precision Trade-off: Float32 vs. INT8 Quantization

Table 2 isolates the wire precision trade-off under a 20% sign-flip attack across 50 communication rounds:

Aggregator	Float32 Acc	INT8 Acc	Delta	F32 Uplink	INT8 Uplink	Cut
FedAvg	40.51 +- 28.25%	41.60 +- 29.28%	+1.09%	397.5 KB	99.4 KB	−75.0%
Multi-Krum	84.89 +- 0.24%	84.90 +- 0.26%	+0.01%	397.5 KB	99.4 KB	−75.0%
Coordinate median	83.21 +- 0.62%	82.97 +- 0.62%	−0.23%	397.5 KB	99.4 KB	−75.0%
Trimmed mean	82.92 +- 0.55%	83.00 +- 0.45%	+0.08%	397.5 KB	99.4 KB	−75.0%

3.2. Learning Dynamics and Convergence Stability

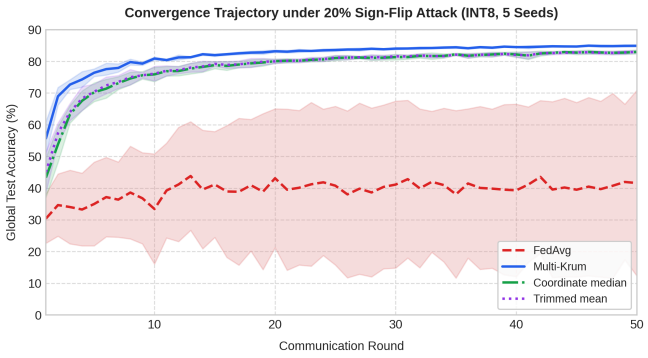


Figure 1: Convergence trajectory under 20% sign-flip attack (INT8, 5 seeds, shaded band = +-1 std).

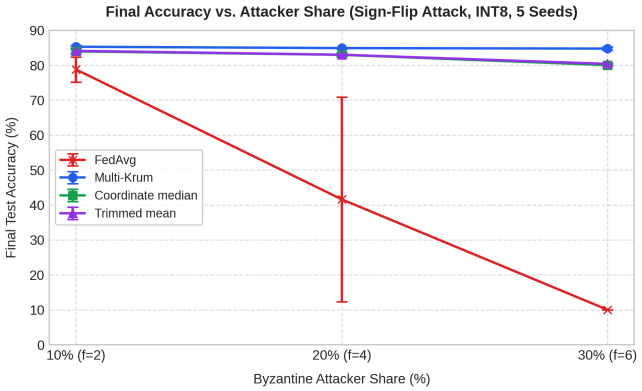


Figure 2: Final test accuracy as a function of Byzantine attacker share (10%, 20%, 30%).

3.3. Vulnerability to Misconfigured Server Assumptions (The Wrong-f Experiment)

Production federated learning servers do not possess oracle knowledge of the true number of compromised edge clients. To evaluate sensitivity to parameter misconfiguration, an extra factorial experiment was executed fixing actual attackers at $f_{\text{act}} = 6$ (30% sign-flip, INT8, 5 seeds) while varying the server's assumed $f_{\text{assumed}} \in \{2, 4, 6, 8\}$:

Assumed f	Assumed Ratio	Multi-Krum Acc	Trimmed Mean Acc	Vulnerability Profile
2	2/20 (10%)	51.65 +- 9.44%	17.61 +- 5.09%	Underestimates attackers: poisoned updates leak into aggregation
4	4/20 (20%)	81.41 +- 3.15%	77.13 +- 3.03%	Partial underestimation: vulnerable to coordinated shift
6	6/20 (30%)	84.75 +- 0.48%	80.37 +- 0.30%	Exact configuration: optimal rejection envelope
8	8/20 (40%)	84.25 +- 0.90%	80.08 +- 0.26%	Overestimates attackers: trims honest samples without collapsing

Critical Architectural Vulnerability: Asymmetric Risk of Misconfigured f

Underestimating f hurts most: with assumed f = 2 and actual f = 6, Multi-Krum falls to 51.65% and Trimmed Mean to 17.61%, because attacker vectors reach the aggregate. Overestimating (assumed f = 8) keeps Multi-Krum at 84.25% and Trimmed Mean at 80.08%, at the cost of discarding extra honest updates. Setting assumed f at or above the expected attacker count is the safer choice in this setup.

3.4. Multi-Krum Isolation and False-Rejection Dynamics

To evaluate whether non-IID label skew causes honest client updates to be falsely rejected as adversaries, Table 4 reports the Attacker Rejection Rate ($|B \cap R| / |B|$) and Honest False-Rejection Rate ($|H \cap R| / |H|$) averaged across all 50 rounds and 5 seeds:

Attack Vector	Attacker Share	Attacker Rejection Rate	Honest False-Rejection Rate
Gaussian	10%	100.00 +- 0.00%	0.00 +- 0.00%
Gaussian	20%	100.00 +- 0.00%	0.00 +- 0.00%
Gaussian	30%	100.00 +- 0.00%	0.00 +- 0.00%
sign-flip	10%	99.80 +- 0.42%	0.02 +- 0.05%
sign-flip	20%	90.80 +- 11.66%	2.30 +- 2.92%
sign-flip	30%	95.50 +- 4.74%	1.93 +- 2.03%

4. System Limitations

To ensure rigorous academic grounding, the scope and boundaries of this benchmark are explicitly stated:

1. **Simulation Environment:** All experiments execute on a single physical host (13th Gen Intel Core i7-13620H) utilizing virtualized client processes rather than physical embedded IoT or mobile hardware. Real-world network jitter, asymmetric packet drop rates, and edge device battery thermal throttling are not modeled.
2. **Model and Dataset Complexity:** Experiments utilize a compact Multi-Layer Perceptron (101,770 parameters) trained on Fashion-MNIST. Deep convolutional architectures (e.g., ResNet) or Vision Transformers over larger feature dimensions may exhibit different gradient sparsity characteristics.
3. **Server Oracle Knowledge:** In the main 280-run grid, the central coordinator is parameterized with the exact true attacker count f, representing an upper bound on defense capability. The sensitivity to unknown f is explored strictly in the dedicated wrong-f experiment (Section 3.3).
4. **Privacy Guarantees:** The benchmark evaluates robust aggregation and gradient compression in isolation. It does not integrate Differential Privacy (DP) mechanisms (e.g., Gaussian gradient clipping and noise addition), which may introduce compound noise interactions with INT8 quantization.

5. Reproduction Instructions

The benchmark is fully automated, self-contained, and reproducible from a single command:

```
# 1. Clone repository and install pinned dependencies
git clone https://github.com/Rajchhapariya/edgefed-bench.git
cd edgefed-bench
pip install -r requirements.txt

# 2. Run comprehensive unit test suite (26 unit tests across 5 test suites)
python -m unittest discover tests -v

# 3. Execute full 280-run factorial grid and wrong-f experiment into DuckDB
python experiments/run_grid.py --config experiments/config.yaml

# 4. Generate all empirical figures, Markdown tables, and Typst assets
python experiments/make_report_assets.py

# 5. Compile 4-page PDF technical report via Typst compiler
typst compile report/edgefed_bench_v2.typ report/edgefed_bench_v2.pdf
```

6. Selected References

1. **Blanchard, P., El Mhamdi, E. M., Guerraoui, R., & Stainer, J. (2017).** Machine learning with adversaries: Byzantine tolerant gradient descent. *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30.
2. **McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017).** Communication-efficient learning of deep networks from decentralized data. *AISTATS 2017*, 1273-1282.
3. **Konečný, J., McMahan, H. B., Yu, F. X., Richtárik, P., Suresh, A. T., & Bacon, D. (2016).** Federated learning: Strategies for improving communication efficiency. *arXiv preprint arXiv:1610.05492*.
4. **Bernstein, J., Wang, Y. X., Azizzadenesheli, K., & Anandkumar, A. (2018).** signSGD: Compressed optimisation for non-convex problems. *ICML 2018*, 560-569.
5. **Raasveldt, M., & Mühleisen, H. (2019).** DuckDB: an embeddable analytical database. *SIGMOD 2019*, 1981-1984.